RAILWAY FARE AND RESERVATION CALCULATOR

Python Programming Case Study

1. Problem Statement

Calculating railway ticket fares manually can be time-consuming when a booking includes multiple passengers. The fare may vary based on travel distance, class of travel, passenger age, reservation charges, and cancellation. A simple program is needed to calculate each passenger's fare and display the total bill.

2. Objective

To develop a Python program that accepts passenger details, calculates the fare using distance slabs and class selection, applies a senior-citizen concession, adds a reservation fee, applies a cancellation adjustment when requested, and displays the total amount for all passengers.

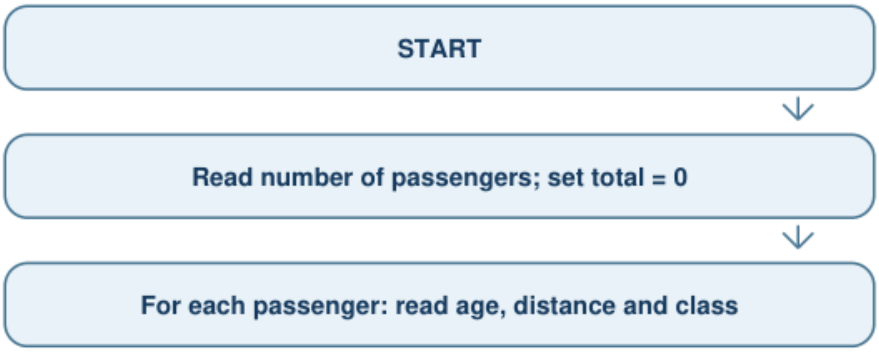
3. Assumptions

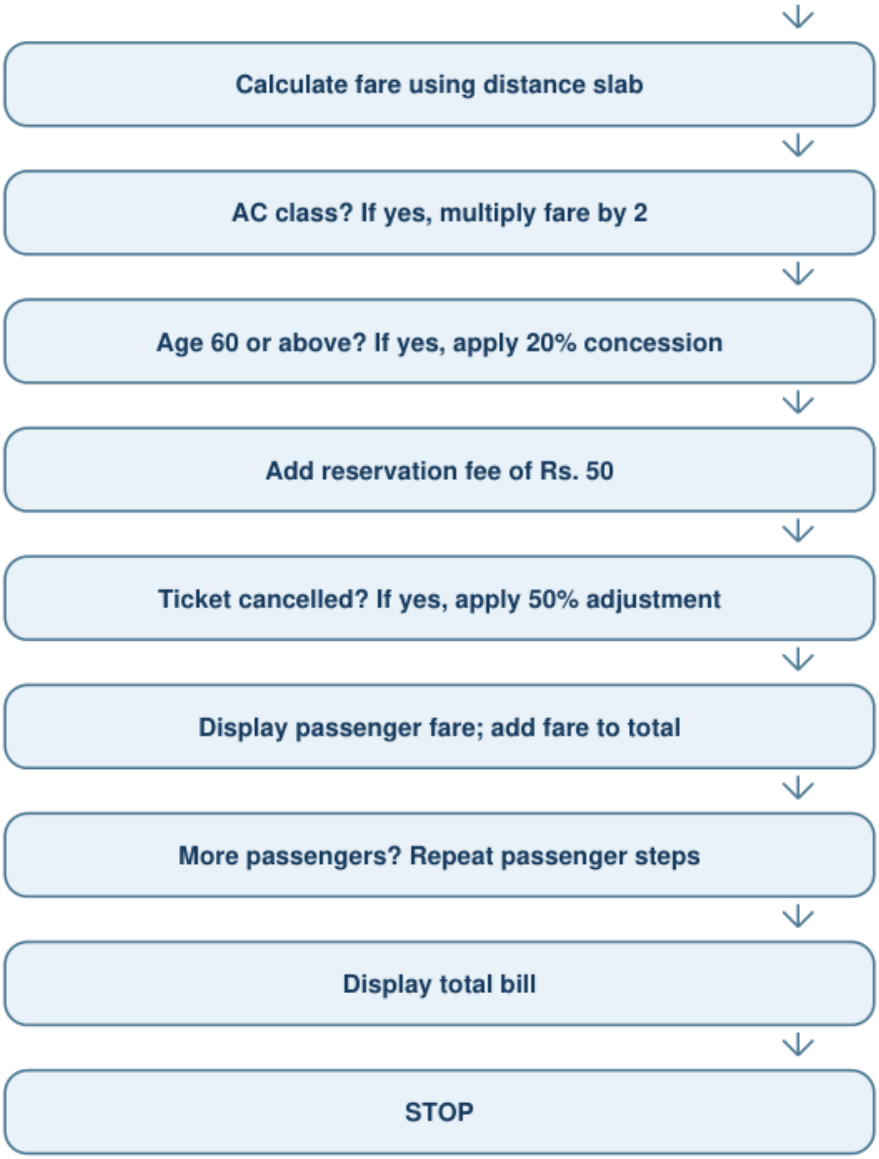
- The user enters a valid number of passengers, age, distance, and travel class.
- Distance is entered in kilometres and is a positive value.
- Sleeper class uses the base fare; AC class doubles the calculated distance fare.
- Passengers aged 60 or above receive a 20% concession on the fare before the reservation fee.
- A reservation fee of Rs. 50 is added for each passenger.
- If the user selects ticket cancellation, the program applies the 50% adjustment shown in the code.

4. Algorithm

1. Start the program and read the number of passengers.
2. Set the total bill to zero.
3. For each passenger, read age, distance, and travel class.
4. Calculate the distance fare: Rs. 1/km for distance below 100 km, Rs. 1.5/km for distance below 500 km, otherwise Rs. 2/km.
5. If the class is AC, multiply the distance fare by 2.
6. If age is 60 or above, apply a 20% concession.
7. Add the Rs. 50 reservation fee.
8. Ask whether the ticket is cancelled; if yes, apply the 50% adjustment.
9. Display the passenger fare and add it to the total bill.
10. After all passengers are processed, display the total amount and stop.

5. Flowchart



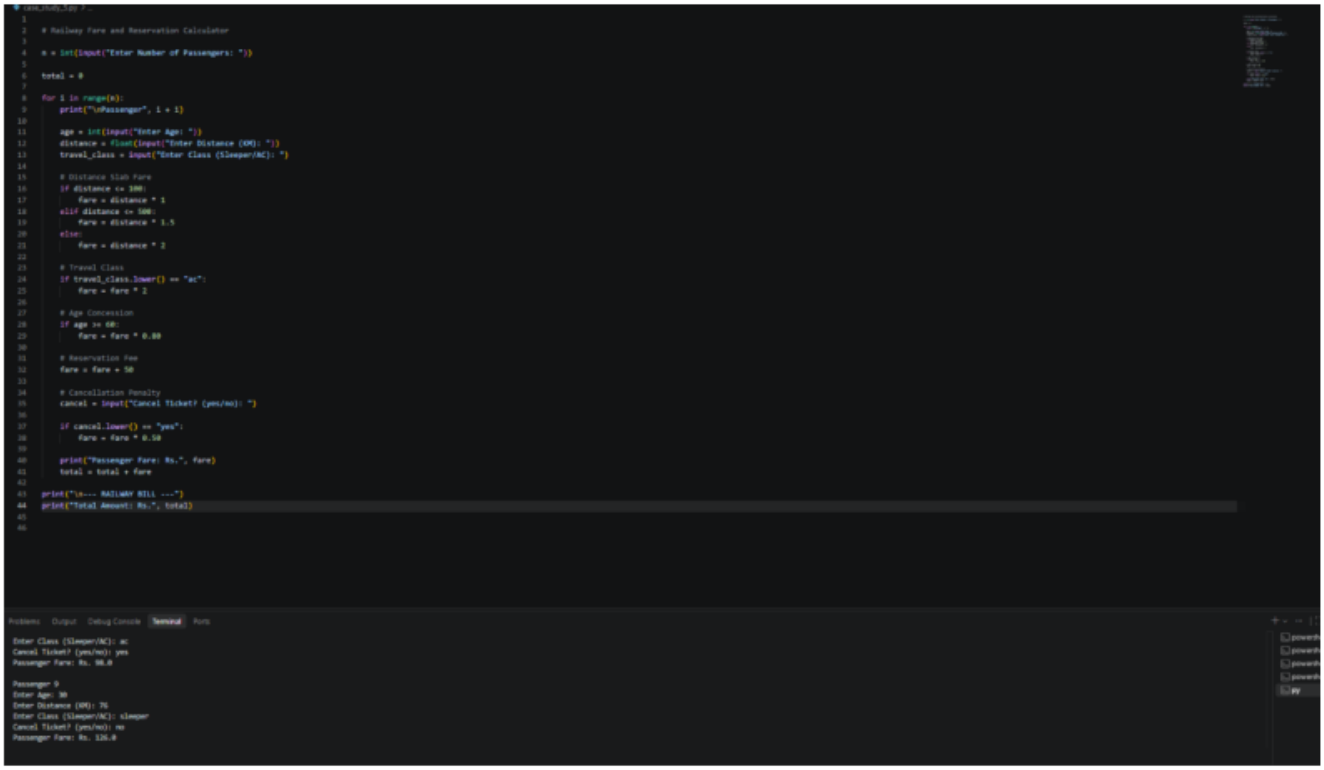

Calculate fare using distance slab
AC class? If yes, multiply fare by 2
Age 60 or above? If yes, apply 20% concession
Add reservation fee of Rs. 50
Ticket cancelled? If yes, apply 50% adjustment
Display passenger fare; add fare to total
More passengers? Repeat passenger steps
Display total bill
STOP

6. Pseudocode

```
BEGIN
INPUT number_of_passengers
total = 0
FOR each passenger
INPUT age, distance, travel_class
IF distance < 100: fare = distance * 1
ELSE IF distance < 500: fare = distance * 1.5
ELSE: fare = distance * 2
IF travel_class is AC: fare = fare * 2
IF age >= 60: fare = fare * 0.80
fare = fare + 50
INPUT cancellation choice
IF cancelled: fare = fare * 0.50
DISPLAY passenger fare
total = total + fare
END FOR
DISPLAY total amount
END
```

7. Python Source Code and Output Screenshot

The original screenshot provided is included below as evidence of the program code and its sample execution.



```
# Railway Fare and Reservation Calculator

n = int(input("Enter Number of Passengers: "))

total = 0

for i in range(n):
    print("\nPassenger", i + 1)

    age = int(input("Enter Age: "))
    distance = float(input("Enter Distance (KM): "))
    travel_class = input("Enter Class (Sleeper/AC): ")

    # Distance Slab Fare
    if distance <= 100:
        fare = distance * 1
    elif distance <= 500:
        fare = distance * 1.5
    else:
        fare = distance * 2

    # Travel Class
    if travel_class.lower() == "ac":
        fare = fare * 1

    # Age Concession
    if age >= 60:
        fare = fare * 0.80

    # Reservation Fee
    fare = fare + 50

    # Cancellation Penalty
    cancel = input("Cancel Ticket? (yes/no): ")

    if cancel.lower() == "yes":
        fare = fare * 0.50

    print("Passenger Fare: Rs.", fare)
    total = total + fare

print("\n--- RAILWAY BILL ---")
print("Total Amount: Rs.", total)
```

```
Enter Class (Sleeper/AC): ac
Cancel Ticket? (yes/no): yes
Passenger Fare: Rs. 98.0

Passenger 9
Enter Age: 30
Enter Distance (KM): 76
Enter Class (Sleeper/AC): sleeper
Cancel Ticket? (yes/no): no
Passenger Fare: Rs. 126.0
```

8. Conclusion

The Railway Fare and Reservation Calculator automates the calculation of passenger fares based on distance, travel class, age concession, reservation fee, and cancellation choice. It processes multiple passengers, shows each passenger's fare, and produces a final total bill. This case study demonstrates the use of input statements, loops, conditional statements, arithmetic operations, and cumulative totals in Python.